VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018

Mini Project Report
On

KrishiCore AI Tagline: "AI & IoT Powered Smart Agriculture Ecosystem"

Submitted in partial fulfillment of the requirements for the
First Semester of the Bachelor of Engineering Degree, towards the completion of the
Mini Project under **the Interdisciplinary Project Based learning,**
Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	ROHAN P	1CR25CS144
2	RIDDHIMA DWIVEDI	1CR25CS143
3	YAHODHA AJIT HEDDURE	1CR25AI145
4	YASHAS V KUMAR	1CR25AI144

Under the Guidance of
Asst. Professor Dr. Puneeth Kumar
Dept. of CAED

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES

CERTIFICATE

This is to certify that the File Structures mini project entitled **KrishiCore AI Tagline: "AI & IoT Powered Smart Agriculture Ecosystem"** has been successfully carried out ROHAN P(1CR25CS144), RIDDHIMA DWIVEDI(1CR25CS143), YASHODHA AJIT HEDDURE(1CR25AI145) ,YASHAS V KUMAR(1CR25AI144), bonafide students of **CMR Institute of Technology.**

The project is submitted in partial fulfillment of the requirements for the First Semester of the Bachelor of Engineering Degree, towards the completion of the Mini Project under the **Interdisciplinary Project Based learning, Department of Basic Sciences.**

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

----------	**----------**
Signature of Guide	**Signature of HOD**
Asst. Professor	**Dr. Raveesha**
Dr. Puneeth Kumar	**HOD,**
Dept. of CAED	**Dept. of Physics, CMRIT**

External Viva

Name of the examiners	Signature with date
1.	
2.	

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H**, **HOD, Dept. of Physics, CMRIT** for his valuable guidance and support.

I am especially grateful to my internal guide, Asst. Prof Dr. PUNEETH KUMAR department of CAED for his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

ROHAN P(1CR25CS144),

RIDDHIMA DWIVEDI(1CR25CS143),

YASHODHA AJITR HEDDURE(1CR25AI145),

YASHAS V KUMAR(1CR25AI144)

ABSTRACT

KrishiCore AI is a modern smart agriculture platform designed to help Indian farmers make better farming decisions using Artificial Intelligence and Internet of Things technologies. The platform combines AI-based crop recommendations, weather forecasting, live market prices, disease detection, and IoT sensor monitoring into one user-friendly web application.

The system uses React and TypeScript for the frontend and FastAPI with Python for the backend. AI functionalities are powered using Groq API with the Llama 3.3 70B model. The platform also uses ESP32 hardware with DHT22 and soil moisture sensors for collecting real-time farm data.

The platform supports both Hindi and English languages and includes a voice-enabled AI assistant that allows farmers to interact using speech. Farmers can upload plant leaf images for disease detection, receive crop and fertilizer recommendations, monitor soil moisture, and check mandi prices directly from the application.

The primary objective of KrishiCore AI is to provide affordable smart farming solutions to Indian farmers and improve productivity, reduce water wastage, and support data-driven agriculture.

TABLE OF CONTENT

Chapter 4

Chapter 5

Chapter 6

LIST OF FIGURES

Chapter 1 :

INTRODUCTION

Agriculture is one of the most important sectors in India and plays a major role in the country's economy. A large percentage of the Indian population depends on farming for their livelihood. However, farmers still face many problems such as unpredictable weather conditions, crop diseases, improper irrigation, incorrect fertilizer usage, and lack of real-time agricultural guidance. These problems often lead to low crop productivity and financial losses.

With the advancement of modern technologies such as Artificial Intelligence (AI), Machine Learning (ML), and Internet of Things (IoT), smart agriculture systems are becoming more effective in solving farming-related challenges. These technologies help farmers monitor crop conditions, analyze soil health, detect diseases, and make better farming decisions using real-time data.

KisanCore AI is a smart agriculture platform developed specifically for Indian farmers. The system combines AI-powered crop recommendations, weather forecasting, disease detection, live market prices, and IoT sensor monitoring into a single web application. The platform is designed to be user-friendly and supports both Hindi and English languages for easy accessibility.

The project uses React and TypeScript for frontend development and FastAPI with Python for backend services. IoT sensors such as DHT22 and Soil Moisture Sensor are connected with ESP32 microcontroller to collect real-time farm data. The collected data is analyzed using AI and machine learning models to provide intelligent farming suggestions.

The main aim of KisanCore AI is to help farmers improve productivity, reduce water wastage, detect diseases early, and make data-driven decisions. The platform also includes a voice-enabled AI assistant so that farmers can interact with the system using speech, making it more convenient for rural users.

By integrating AI, IoT, and cloud technologies, KisanCore AI provides an affordable and efficient smart farming solution that can improve the overall quality and sustainability of agriculture in India.

1.1 Brief History:

Traditional farming methods were mainly dependent on manual observation, local knowledge, and the personal experience of farmers. Decisions regarding irrigation, crop selection, and pest control were taken based on intuition and past practice. While these methods were effective in earlier times, they often led to inconsistent yield due to unpredictable weather conditions and limited access to scientific information.

With the advancement of technology, agriculture gradually began to adopt digital tools. Early agricultural software systems were simple and mainly provided weather forecasts, seasonal farming tips, and general advisory information. These systems helped farmers to some extent but were not capable of real-time decision-making or automation.

As technology evolved further, the introduction of Machine Learning (ML) and Internet of Things (IoT) brought a major transformation in agriculture. Sensors started being used to collect real-time data from fields such as soil moisture, temperature, humidity, and crop health. Machine learning algorithms enabled systems to analyse this data and provide predictions related to crop growth, disease detection, and yield estimation. This marked the beginning of smart agriculture systems.

In the present era, Artificial Intelligence (AI)-powered agriculture platforms integrate IoT devices, cloud computing, and advanced analytics to deliver highly accurate and real-time farming insights. These systems assist farmers in making better decisions, improving productivity, and reducing losses caused by environmental and biological factors.

KisanCore AI is developed as a modern smart agriculture solution that brings all these technologies together in a single platform, specifically designed to support Indian farmers with intelligent, data-driven farming assistance.

Chapter 2:

Problem Statement

agriculture in India faces several challenges due to lack of modern technology adoption and limited access to accurate farming information. Most farmers still depend on traditional farming methods and manual monitoring systems. As a result, they often suffer from low productivity, crop diseases, water wastage, and financial losses.

One of the major problems is the absence of real-time monitoring systems. Farmers cannot continuously monitor soil moisture, humidity, or environmental conditions. This leads to improper irrigation and poor crop health.

Another major issue is the lack of expert agricultural guidance. Many rural farmers do not have easy access to agricultural experts who can provide recommendations regarding fertilizer usage, crop selection, disease treatment, Aand irrigation management.

Crop diseases also cause significant damage to agricultural production. Farmers usually identify diseases at later stages when crops are already heavily affected. Manual disease detection is time-consuming and inaccurate.

Weather conditions are also highly unpredictable. Sudden rainfall, temperature changes, or drought conditions directly affect crop productivity. Farmers often do not receive proper weather alerts or farming suggestions.

In addition, farmers face problems related to market prices. Many small farmers sell their crops without knowing current mandi prices, leading to lower profits.

Therefore, there is a strong need for a smart agriculture platform that can provide real-time monitoring, AI-powered recommendations, disease detection, market prices, and weather information in an affordable and user-friendly manner.

2.1 PROBLEM STATEMENT

Indian farmers often face difficulties due to lack of access to expert agricultural advice, real-time weather information, crop disease detection, and market price updates. Many small-scale farmers cannot afford expensive smart farming technologies.

Major problems faced by farmers include:

- Lack of real-time weather updates
- Incorrect crop selection
- Overuse or underuse of fertilizers
- Late detection of crop diseases
- Water wastage due to improper irrigation
- Unavailability of expert guidance
- Lack of knowledge about mandi prices

Therefore, there is a need for an affordable smart agriculture platform that combines AI and IoT technologies to help farmers improve productivity and reduce losses.

2.2 Objective of the Project

The main objectives of KisanCore AI are:

- To provide AI-based crop recommendations
- To monitor real-time farm conditions using IoT sensors
- To provide weather forecasting and farming tips
- To detect crop diseases using image analysis
- To provide live mandi market prices
- To support voice-enabled farming assistance
- To improve water management through smart irrigation suggestions
- To support both Hindi and English languages

2.3 Proposed Solution

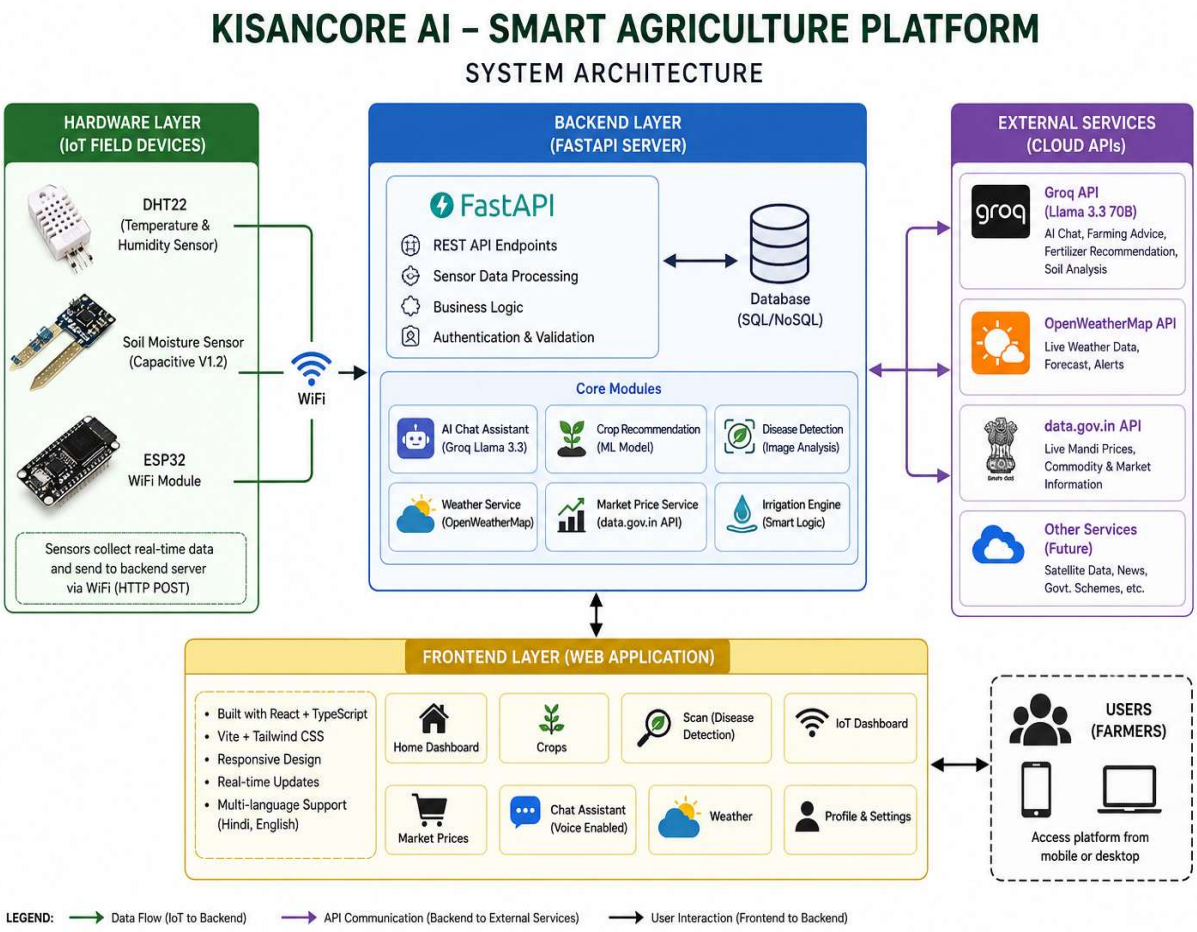
KisanCore AI provides a complete smart agriculture platform using Artificial Intelligence and Internet of Things technologies.

The proposed system includes:

- AI-powered farming assistant
- Real-time weather monitoring
- IoT sensor integration using ESP32
- Crop recommendation system using machine learning
- Disease detection from leaf images
- Smart irrigation suggestions
- Market price monitoring
- Voice interaction system

2.4 Advantages of Proposed Solution

- Advantages of KisanCore AI include:
- User-friendly interface
- Real-time monitoring
- Accurate crop recommendations
- Water conservation through smart irrigation
- Reduced farming losses
- Faster disease detection
- Multilingual support
- Affordable solution for small farmers
- Remote farm monitoring
- Better productivity and crop quality



2.5 System Architecture Diagram

CHAPTER 3

DESIGN:

3.1 Description of Approach to Solve the Problem

The project uses a three-tier architecture consisting of frontend, backend, and hardware layers.

Frontend Layer

The frontend is developed using React, TypeScript, Vite, and Tailwind CSS. It provides a responsive and interactive user interface.

Backend Layer

The backend is developed using FastAPI in Python. It handles API requests, sensor data processing, AI communication, and business logic.

Hardware Layer

ESP32 hardware with DHT22 and soil moisture sensors collects real-time farm data and sends it to the backend server.

The system also integrates external APIs such as:

- Groq API for AI chatbot and recommendations
- OpenWeatherMap API for weather data
- data.gov.in API for mandi market prices

3.2 Methodology

The development methodology of KisanCore AI follows a systematic approach involving data collection, hardware integration, backend processing, AI analysis, and frontend visualization.

Requirement Analysis

During the initial phase, problems faced by farmers were analyzed carefully. The major focus areas identified were irrigation management, disease detection, weather monitoring, crop recommendations, and market price analysis.

System Design

The overall architecture of the system was designed using a three-layer approach:

1. Frontend Layer
2. Backend Layer
3. Hardware Layer

Each layer was developed independently and later integrated into a single platform.

Frontend Development

The frontend interface was designed using React, TypeScript, Vite, and Tailwind CSS. Multiple pages were created such as:

- Home Page
- Crops Page
- Scan Page
- IoT Dashboard
- Market Page
- Chat Assistant Page
- Weather Page

Responsive design techniques were used to ensure compatibility with mobile phones, tablets, and desktop devices.

Backend Development

The backend server was developed using FastAPI in Python. REST APIs were created for:

- Sensor data collection
- Weather information
- Crop recommendation
- AI chatbot communication
- Market price retrieval

- Disease detection

The backend acts as the central processing unit of the system.

IoT Hardware Integration

ESP32 microcontroller was connected with DHT22 and Soil Moisture sensors. The ESP32 continuously reads sensor values and sends them to the backend server through WiFi using HTTP requests.

AI and Machine Learning Integration

Machine learning models were trained using crop and soil datasets. Random Forest Algorithm was used for crop prediction because of its high accuracy and reliability.

The Groq API with Llama 3.3 70B model was integrated for:

- AI farming assistant
- Soil analysis
- Fertilizer recommendations
- Smart farming suggestions

Testing Phase

The system was tested under different environmental conditions to ensure proper functionality. Sensor readings, API responses, and AI recommendations were verified.

Deployment

The final system was integrated into a single smart agriculture platform capable of handling real-time farming operations.

3.2 Methodology

The methodology used in the project includes the following steps:

Step 1: Data Collection

Sensor data such as temperature, humidity, and soil moisture is collected using ESP32 hardware.

Step 2: Backend Processing

The backend server receives sensor data and processes it using APIs and machine learning models.

Step 3: AI Analysis

The AI system analyzes weather, soil, and crop conditions and generates intelligent recommendations.

Step 4: Frontend Display

The frontend displays sensor values, crop recommendations, weather reports, and disease analysis results.

Step 5: User Interaction

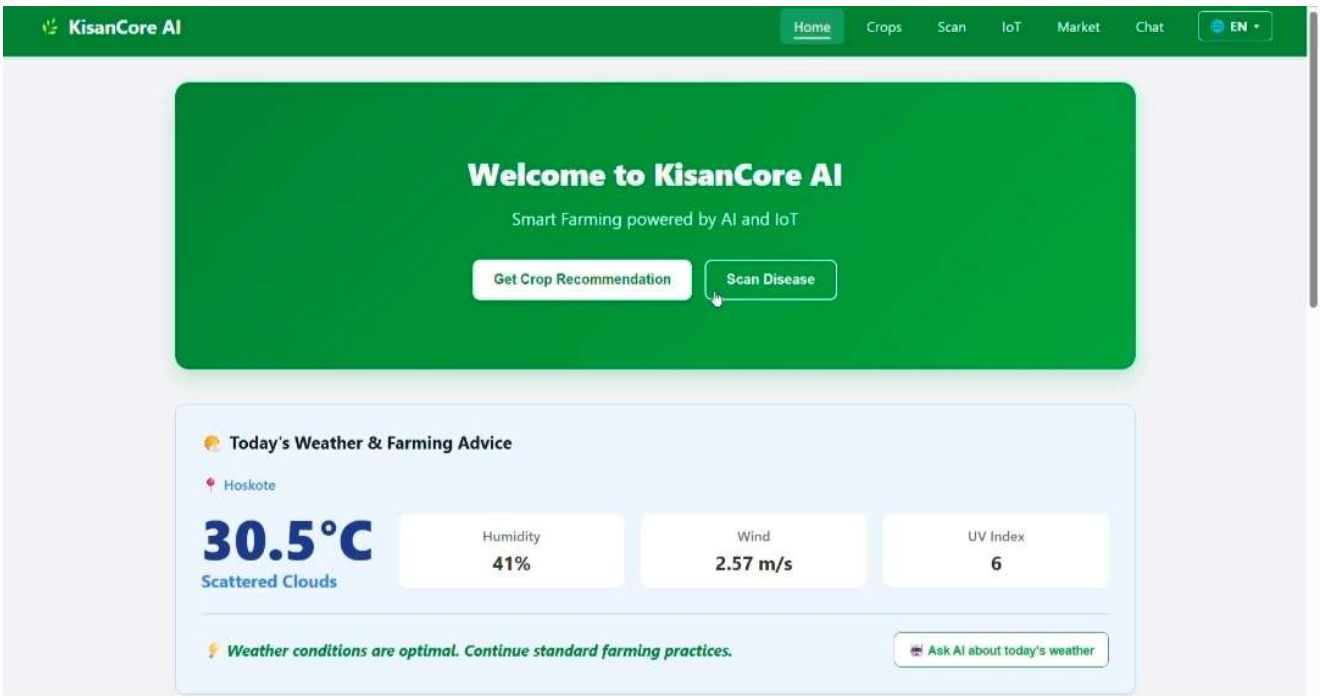
Farmers can interact with the AI assistant through text or voice input.

3.3 Prototype Description

The KisanCore AI prototype contains several modules:

Home Page

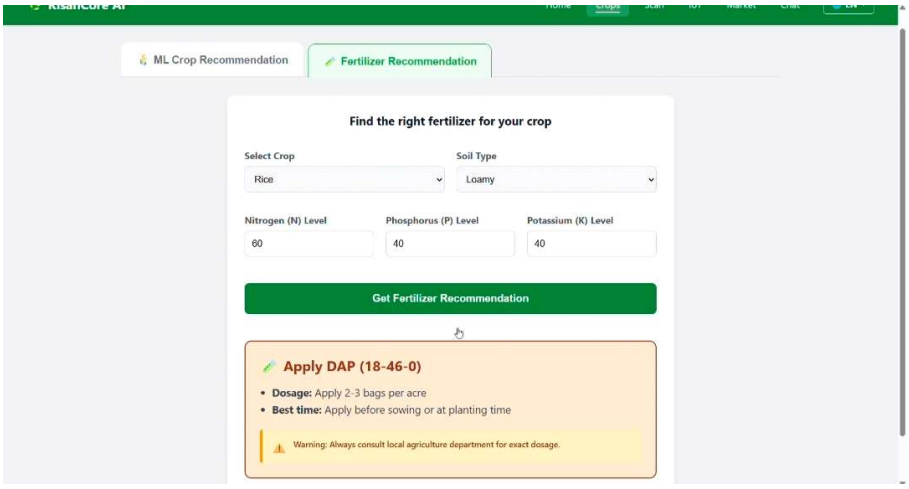
- Live weather information
- Sensor data cards
- AI crop recommendations
- Soil health analyse



2.5 Home page design

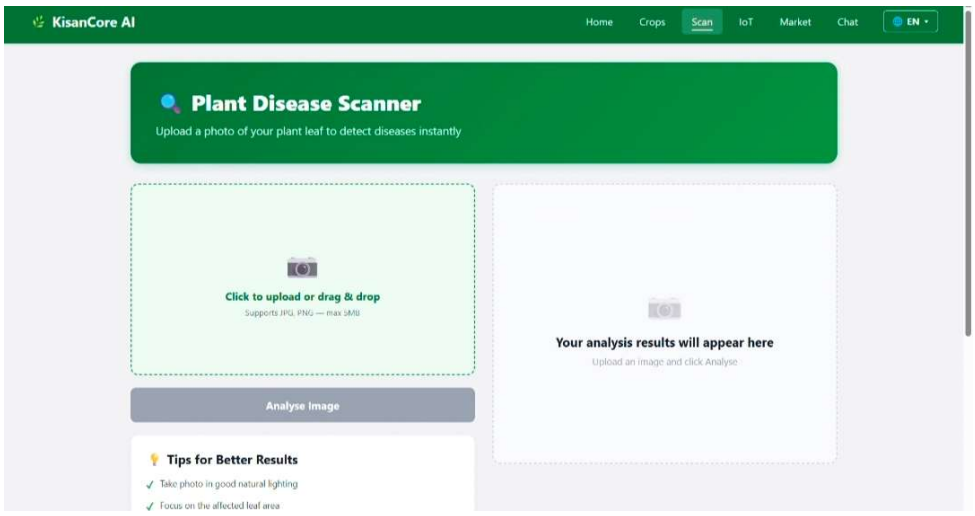
Crops Page

- Crop recommendation using Random Forest ML model
- Fertilizer recommendation system



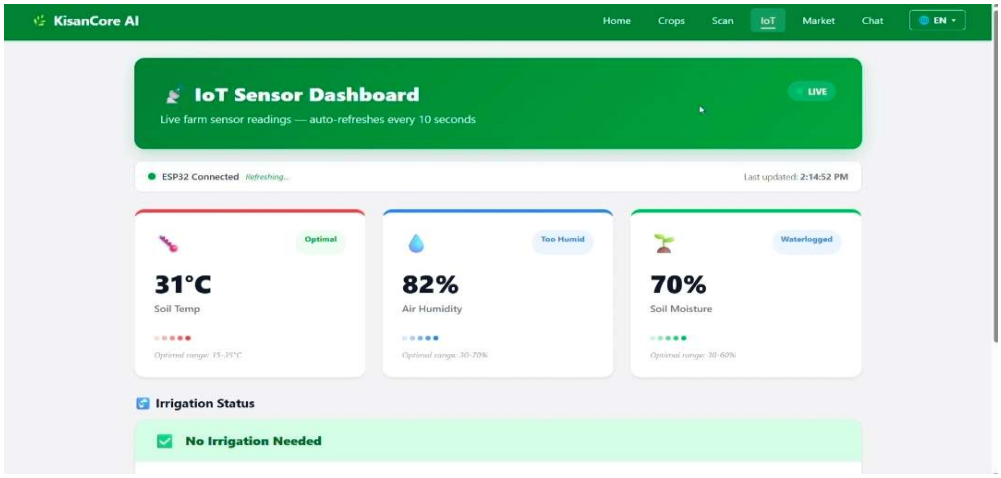
Scan Page

- Upload plant leaf images
- AI disease detection
- Treatment suggestion



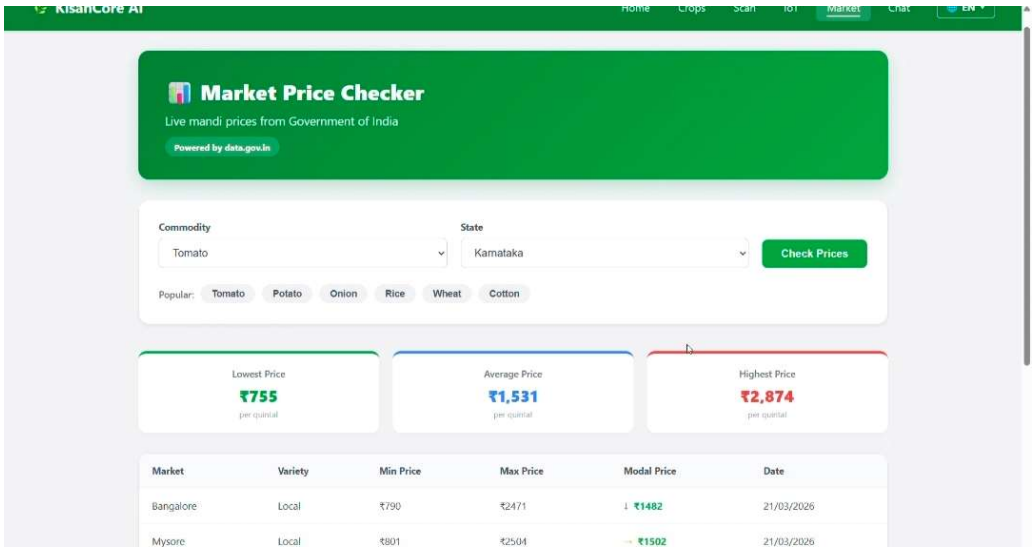
IoT Dashboard

- Real-time temperature monitoring
- Humidity monitoring
- Soil moisture monitoring
- Irrigation status



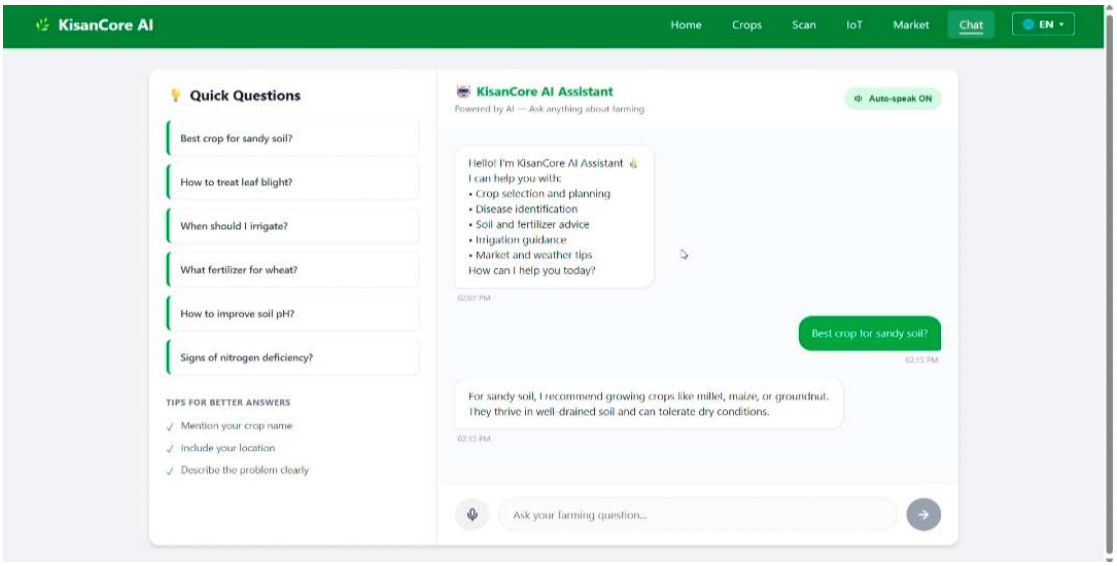
Market Page

- Live mandi prices
- Commodity selection
- State-wise price display



Chat Page

- AI farming assistant
- Voice interaction support
- Multilingual communication



CHAPTER 4

CODE SNIPPETS AND DESCRIPTION

4.1 Generalized Code for KisanCore AI Smart Agriculture Platform

```python
import joblib
from pathlib import Path
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
import warnings

router = APIRouter()

MODEL_PATH = Path(__file__).resolve().parent.parent.parent.parent / "model.pkl"
model = None
if MODEL_PATH.exists():
    try:
        model = joblib.load(MODEL_PATH)
    except Exception as e:
        warnings.warn(f"Failed to load model.pkl: {e}")

class CropRequest(BaseModel):
    N: float = 0.0
    P: float = 0.0
    K: float = 0.0
    temperature: float
    humidity: float
    ph: float
    rainfall: float

def _build_features(N: float, P: float, K: float, temperature: float, humidity: float, ph: float, rainfall: float):
    if hasattr(model, "n_features_in_"):
        n = int(model.n_features_in_)
        if n == 4:
            return [temperature, humidity, ph, rainfall]
        if n == 7:
            return [N, P, K, temperature, humidity, ph, rainfall]
    return [N, P, K, temperature, humidity, ph, rainfall]

@router.get("")
def recommend_crop_get(temperature: float, humidity: float, ph: float, rainfall: float, N: float = 0.0, P: float = 0.0, K: float = 0.0):
    if model is None:
        raise HTTPException(status_code=500, detail="Model not loaded on server")
    features = _build_features(N, P, K, temperature, humidity, ph, rainfall)
    prediction = model.predict([features])[0]
    return {"crop": prediction}

@router.post("")
def recommend_crop_post(payload: CropRequest):
    if model is None:
        raise HTTPException(status_code=500, detail="Model not loaded on server")
    features = _build_features(payload.N, payload.P, payload.K, payload.temperature, payload.humidity, payload.ph, payload.rainfall)
    prediction = model.predict([features])[0]
    return {"crop": prediction}
```

```python
from fastapi import APIRouter
from pydantic import BaseModel
from datetime import datetime, timezone, timedelta
from typing import Optional, List
import time
import os
import json

router = APIRouter()

# --- IST CONFIGURATION ---
IST = timezone(timedelta(hours=5, minutes=30))

# --- MODELS ---
class IOTData(BaseModel):
    temperature: float
    humidity: float
    soil_moisture: float

class OverrideRequest(BaseModel):
    command: str  # "ON", "OFF"
    duration_minutes: Optional[int] = 60
    duration_seconds: Optional[int] = 0

# --- IN-MEMORY STORAGE & PERSISTENCE ---
DATA_FILE = "latest_reading.json"

latest_reading = {
    "temperature": 0.0,
```

```python
import joblib
from pathlib import Path
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
import warnings

router = APIRouter()

MODEL_PATH = Path(__file__).resolve().parent.parent.parent.pa
model = None
if MODEL_PATH.exists():
    try:
        model = joblib.load(MODEL_PATH)
    except Exception as e:
        warnings.warn(f"Failed to load model.pkl: {e}")

class CropRequest(BaseModel):
    N: float = 0.0
    P: float = 0.0
    K: float = 0.0
    temperature: float
    humidity: float
    ph: float
    rainfall: float

def _build_features(N: float, P: float, K: float, temperature
    if hasattr(model, "n_features_in_"):
        n = int(model.n_features_in_)
```

```yaml
services:
  - type: web
    name: kisancore-api
    env: python
    region: singapore
    plan: free
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
    disk:
      name: kisancore-data
      mountPath: /data
      sizeGB: 1
    envVars:
      - key: PYTHON_VERSION
        value: 3.11
      - key: GROQ_API_KEY
        sync: false
      - key: OPENWEATHER_API_KEY
        sync: false
      - key: TWILIO_ACCOUNT_SID
        sync: false
      - key: TWILIO_AUTH_TOKEN
        sync: false
      - key: TWILIO_WHATSAPP_FROM
        sync: false
      - key: SECRET_KEY
        generateValue: true
      - key: DATABASE_URL
        value: sqlite:////data/kisancore.db
```

```javascript
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    port: 5173,
    host: true, // Allow Mobile connections
    proxy: {
      "/api": {
        target: "http://127.0.0.1:8000",
        changeOrigin: true
      }
    }
  }
});
```

Chapter 5

RESULT AND ANALYSIS

5.1 Performance Analysis

The performance of KisanCore AI was analyzed based on response time, accuracy, usability, reliability, and hardware efficiency.

Sensor Performance

The DHT22 sensor provided accurate temperature and humidity readings with stable performance. The soil moisture sensor successfully monitored water content in the soil and updated values continuously.

Backend Performance

FastAPI backend provided high-speed API responses with low latency. The backend handled multiple API requests efficiently.

AI Performance

The AI assistant generated farming responses quickly using the Groq API. Crop recommendations and fertilizer suggestions were generated accurately based on user input and sensor values.

Disease Detection Performance

The image upload and disease detection module successfully identified common plant diseases and provided treatment recommendations.

User Interface Performance

The React frontend provided smooth navigation and responsive design. Real-time updates were displayed without significant delay.

Overall System Performance

The integration between frontend, backend, AI modules, and IoT hardware worked successfully. Real-time sensor monitoring and AI recommendations improved the efficiency of the platform.

Key Observations

- Real-time sensor updates every 5 seconds
- Fast AI response generation

- Smooth communication between ESP32 and backend
- Accurate crop recommendation results
- Stable weather API integration
- User-friendly multilingual interface

5.2 Performance Analysis

The system provides fast response times and accurate recommendations.

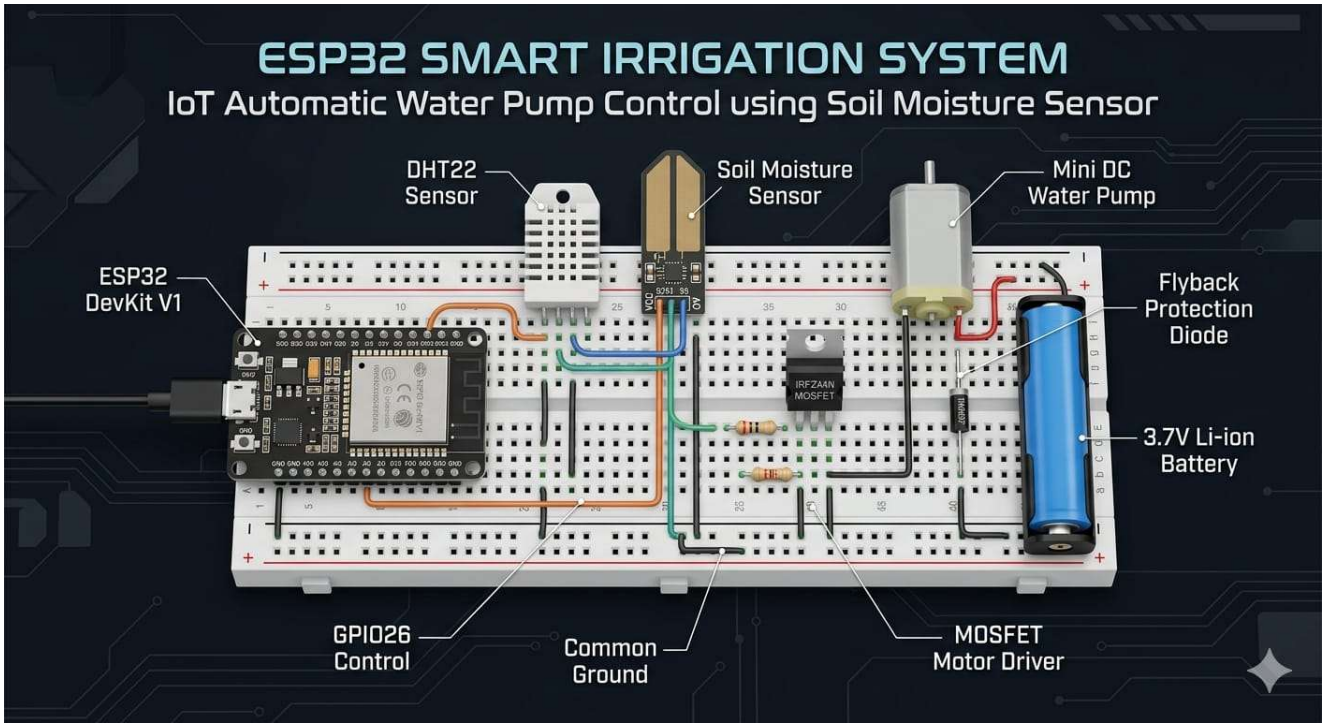
Key Results

- Real-time sensor updates every 5 seconds
- Accurate weather forecasting
- Fast AI response generation
- Successful disease detection from leaf images
- Stable communication between ESP32 and backend server

5.3 Advantages to Farmers

Benefits of the system include:

- Better farming decisions
- Improved crop productivity
- Reduced fertilizer misuse
- Water conservation
- Early disease detection
- Easy access to agricultural information
- Voice-based assistance for rural farmers



5.4 ESP32 SMART IRRIGATION STSTEM

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 Conclusion

KRISHICORE AI is an innovative smart agriculture platform developed to support Indian farmers using Artificial Intelligence (AI) and Internet of Things (IoT) technologies. The project successfully integrates multiple smart farming features such as weather forecasting, crop recommendation, disease detection, live market prices, smart irrigation, and real-time sensor monitoring into a single platform.

The system helps farmers make data-driven farming decisions and improves agricultural productivity while reducing water wastage and farming losses. The integration of IoT sensors with AI analysis enables real-time monitoring of environmental conditions and provides intelligent farming suggestions.

The platform also includes a voice-enabled AI assistant and multilingual support in Hindi and English, making it easy to use for rural farmersKRISHICORE AI provides an affordable, user-friendly, and efficient smart farming solution that promotes modern and sustainable agricultural practices.

Overall, the project demonstrates how AI, machine learning, cloud computing, and IoT technologies can be used together to improve the quality and efficiency of agriculture in India.

6.2 Future Enhancements

Future improvements that can be added to the project include:

- Mobile application development
- Drone integration for field monitoring
- Satellite-based crop analysis
- Advanced deep learning disease detection
- SMS alert system for farmers
- Offline mode for rural areas with poor internet

REFERENCES

1. React Official Documentation – https://react.dev
2. FastAPI Documentation – https://fastapi.tiangolo.com
3. OpenWeatherMap API – https://openweathermap.org/api
4. Groq API Documentation – https://groq.com
5. data.gov.in API – https://data.gov.in
6. Scikit-learn Documentation – https://scikit-learn.org
7. ESP32 Documentation – https://www.espressif.com
8. Tailwind CSS Documentation – https://tailwindcss.com
9. Vite Official Documentation – https://vitejs.dev
10. Arduino IDE Documentation – https://www.arduino.cc/en/software